



DOCUMENTO TÉCNICO

ANÁLISIS TÉCNICO Y ARQUITECTURA DEL SISTEMA

Sistema de Análisis Facial Vectorial en Vivo — BeatC

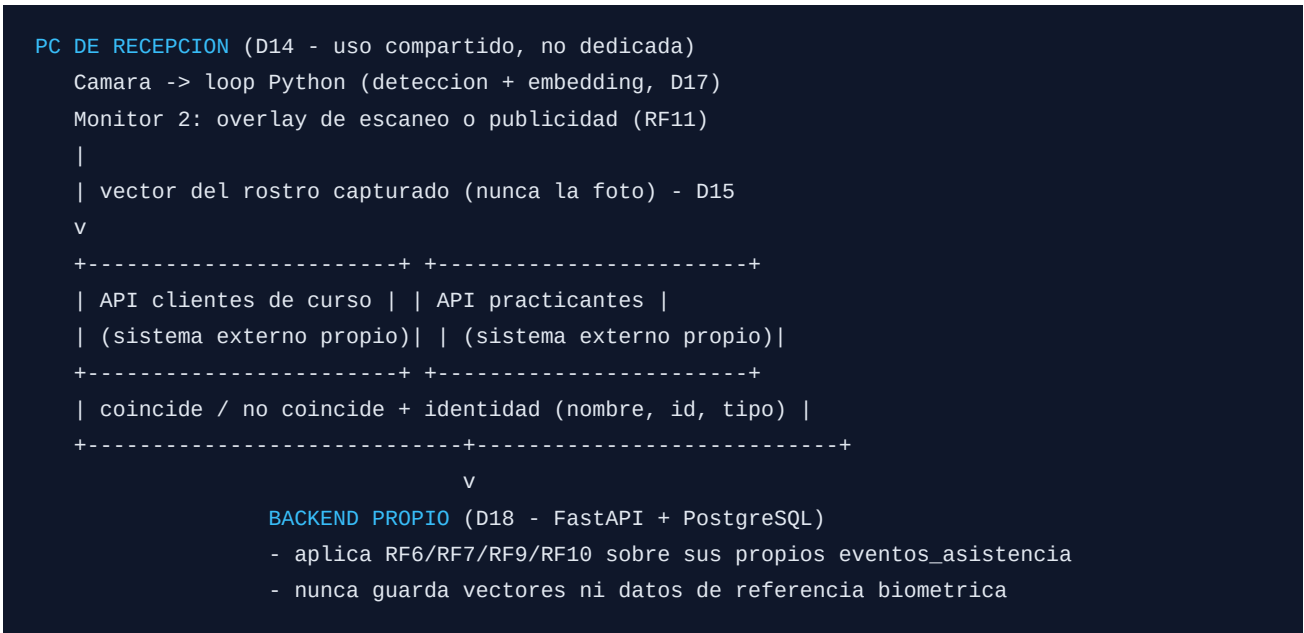
CONTENIDO

1. Arquitectura general y flujo de datos
2. Justificación del stack tecnológico
3. Motor de reconocimiento facial — detalle técnico
4. Máquina de estados del kiosco
5. Modelo de datos propuesto
6. Diseño de API: backend propio y verificación externa
7. Seguridad, privacidad y resiliencia
8. Hardware y requisitos no funcionales
9. Alcance: qué NO determina este sistema
10. Pendientes técnicos explícitos

1. ARQUITECTURA GENERAL Y FLUJO DE DATOS

BLOQUEANTE (confirmado con el equipo): la conexión real con los 2 sistemas externos (clientes de curso y practicantes) todavía no está especificada. No se avanza en construir esta integración hasta tener sus especificaciones reales — lo que sigue es el diseño propuesto, sujeto a lo que esas APIs realmente permitan hacer.

El sistema corre en la **PC de recepción** (D14) — una computadora de uso general, no un equipo dedicado — y se comunica exclusivamente por HTTPS/REST con tres sistemas distintos, cada uno con una responsabilidad acotada: el **motor local** ejecuta el reconocimiento facial y nunca abre puertos de entrada; los **2 sistemas externos** (D16) son quienes verifican el vector capturado contra su propia base de personas, sin que este sistema la vea jamás (D15); el **backend propio** (D18) solo guarda los eventos de entrada/salida que este proyecto genera.



Precisión de diseño respecto al documento de contexto: el motor local no decide si un reconocimiento exitoso es "entrada" o "salida", ni si corresponde al cooldown de RF9 o al límite diario de RF10 — solo reporta *"persona X reconocida, identidad Y"*. Es el backend propio quien aplica esas reglas de negocio contra el estado real en su base de datos (D11/D18). Como no hay caché local de referencia (D15), sin conexión a las 2 APIs externas este sistema no puede identificar a nadie — ver sección 7.

2. JUSTIFICACIÓN DEL STACK TECNOLÓGICO

Se evaluó cada alternativa contra 5 criterios, no solo "qué ya se conoce":

Opción (motor facial)	Rendimiento	Madurez/ecosistema	Curva de aprendizaje	Licencia	Veredicto
Python + OpenCV (YuNet + SFace)	Alto en CPU x86 para 1 rostro a la vez	Alta — ya validado en proyecto previo del equipo	Baja — código reutilizable	OpenCV (Apache 2.0), sin restricción comercial	Elegida

Opción (motor facial)	Rendimiento	Madurez/ecosistema	Curva de aprendizaje	Licencia	Veredicto
MediaPipe / InsightFace / DeepFace	Variable, requiere re-validación	Alta en general, pero sin historial propio aquí	Media-alta (nueva integración)	Algunas variantes con licencias no comerciales	Descartada
C++ nativo con OpenCV	El más alto en teoría	Alta (librería madura)	Alta — sin base de código ni experiencia del equipo	Apache 2.0	Descartada (costo/beneficio)
Node.js (face-api.js, onnxruntime-node)	Medio-bajo	Baja para visión por computadora	Media	Variable según modelo	Descartada

Backend propio: FastAPI + PostgreSQL, por continuidad directa con la experiencia del equipo (ORM, migraciones, RBAC, auditoría ya resueltos en proyectos anteriores). Guarda únicamente eventos de entrada/salida (D18) — nunca datos de referencia biométrica.

D17 (confirmado por el equipo): BeatC exige PHP para su ecosistema, pero este sistema está aislado — solo se comunica por API, nunca comparte código, proceso ni base de datos con el resto. Eso hace que el lenguaje detrás de esa API sea invisible para quien la consume: sin problema de compatibilidad. El motor de reconocimiento se construye en Python (esta tabla), el resto del ecosistema de BeatC permanece en PHP.

3. MOTOR DE RECONOCIMIENTO FACIAL — DETALLE TÉCNICO

Detección — YuNet

Modelo ONNX liviano (≈340 KB) de detección facial. Entrada: fotograma BGR. Salida: cuadro delimitador (bounding box) + 5 puntos de referencia faciales (ojos, nariz, comisuras de la boca) + puntaje de confianza por rostro detectado — permite alinear el rostro antes de pasarlo al siguiente paso.

Reconocimiento — SFace

A partir del rostro alineado y recortado (112×112 px), SFace produce un **vector de 128 dimensiones** (embedding) que representa numéricamente ese rostro. El vector se normaliza (norma L2 = 1) antes de compararlo.

Decisión de coincidencia

```
similitud = (vector_vivo · vector_guardado) / (||vector_vivo|| · ||vector_guardado||)

# al estar ambos normalizados (norma L2 = 1), esto equivale al producto punto directo:
similitud = vector_vivo · vector_guardado

coincide = similitud >= UMBRAL_SIMILITUD
```

Umbral de referencia: 0.363 — valor validado en un despliegue anterior del equipo con el mismo motor (YuNet+SFace). Se mantiene como punto de partida, pero debe **recalibrarse con fotos reales del entorno de BeatC** (iluminación, distancia y cámara específicas) antes de producción — ver sección 9.

Muestreo de video: no es necesario procesar los 30 FPS típicos de una cámara — se recomienda muestrear entre 5 y 10 FPS para el pipeline de detección+comparación, suficiente para sostener con precisión el cronómetro de

enfoque de RF5 sin saturar la CPU del kiosk.

4. MÁQUINA DE ESTADOS DEL KIOSCO

Estado	Condición de entrada	Condición de salida
ESPERA	Inicio, o resultado ya mostrado 3s	Se detecta un rostro en un frame muestreado
ENFOCANDO	Rostro detectado	5s continuos → ANALIZANDO · rostro desaparece → ESPERA
ANALIZANDO	5s de enfoque completados	Similitud calculada → MOSTRANDO_RESULTADO
MOSTRANDO_RESULTADO	Resultado disponible (ver los 3 casos abajo)	Pasan 3s → ESPERA

MOSTRANDO_RESULTADO ya no es binario (coincide / no registrado) — RF9 y RF10 agregan un tercer caso.

El backend, al recibir el reconocimiento (D11), responde con uno de tres resultados, y el kiosk muestra el mensaje correspondiente:

Resultado	Qué muestra el monitor 2
Coincide, evento nuevo guardado	Nombre, DNI, tipo de evento (entrada/salida) y fecha/hora (RF6)
Coincide, pero en cooldown (RF9)	Mensaje distinto: "Ya registrado, intente su salida después de las HH:MM" — nunca "no registrado"
Coincide, pero límite diario alcanzado (RF10)	"No registrado" — igual que un rostro desconocido (RF4), de forma deliberada
Ninguna de las 2 APIs externas reconoce el vector	"No registrado" (RF4)

RF11 — modo publicidad: el estado ESPERA de arriba, en el monitor 2, no muestra una pantalla vacía — reproduce videos publicitarios de BeatC en loop, a pantalla completa. En cuanto hay un rostro (transición a ENFOCANDO), la pantalla cambia a pantalla completa de escaneo; al volver a ESPERA, retoma la publicidad. Confirmado: alternancia a pantalla completa, no división fija de pantalla.

```

focus_timer = 0
while True:
    frame = camara.leer() # muestreo ~5-10 FPS
    rostro = detectar_mas_prominente(frame) # D10: el de mayor tamaño

    if rostro is None:
        focus_timer = 0; estado = ESPERA; mostrar_publicidad(); continue # RF11

    focus_timer += intervalo_muestreo
    if focus_timer >= 5.0:
        vector = sface.embedding(rostro)
        respuesta = verificar(vector) # llama a las 2 APIs externas (D15/D16) - BLOQUEANTE
        registrar_evento(respuesta) # backend propio aplica RF9/RF10 (D18)
        mostrar(respuesta.mensaje); dormir(3.0) # ver tabla de 3 resultados arriba

```

```
focus_timer = 0
```

5. MODELO DE DATOS PROPUESTO

Esquema final tras D18 — reemplaza un esbozo anterior que incluía una tabla personas con un campo de embedding propio. Eso violaba D15 (este sistema nunca guarda datos de referencia biométrica), así que se eliminó: no existe tabla personas propia.

```
eventos_asistencia
  id serial PK
  persona_externa_id varchar NOT NULL -- id que devuelve el sistema externo al verificar
  tipo_persona enum('practicante', 'cliente_curso') NOT NULL
  nombre varchar(120) NOT NULL -- copiado de la respuesta externa
  tipo_evento enum('entrada', 'salida') NOT NULL
  marca_tiempo timestamptz NOT NULL -- calculada en el servidor
  cierre_automatico boolean NOT NULL DEFAULT false
  creado_en timestamptz NOT NULL DEFAULT now()

INDEX idx_eventos_persona_fecha (persona_externa_id, marca_tiempo DESC)
-- permite responder rapido: "cual fue el ultimo evento de esta persona hoy?"
```

persona_externa_id + tipo_persona identifican a la persona usando el identificador que ya devuelve el sistema externo al verificar — este backend nunca es la fuente de verdad de quién es cada persona, solo de cuándo entró y salió. **RF9 y RF10 no requieren columnas nuevas** — el mismo índice de arriba alcanza para ambas reglas: el cooldown de RF9 se resuelve comparando `now() - ultimo_evento.marca_tiempo` contra 30 minutos, y el límite de RF10 con un `COUNT(*)` de eventos de esa persona en el día.

6. DISEÑO DE API: BACKEND PROPIO Y VERIFICACIÓN EXTERNA

Backend propio (D18) — lo que sí controla este proyecto

Método y ruta	Quién la llama	Propósito
POST /api/eventos	Motor local	Reporta una identidad ya verificada por las APIs externas; el backend decide entrada/salida y aplica RF9/RF10
GET /api/asistencia	Panel administrativo (futuro)	Consulta de eventos para reportes

Respuesta de POST /api/eventos (los 3 casos de RF9/RF10)

```
// caso 1: evento nuevo guardado
{"resultado": "registrado", "tipo": "entrada", "nombre": "...", "marca_tiempo": "..."}

// caso 2: en cooldown (RF9) - el backend calcula cuando puede volver a registrarse
{"resultado": "cooldown", "disponible_desde": "2025-01-01T09:30:00Z"}

// caso 3: limite diario alcanzado (RF10) - tratado igual que RF4
{"resultado": "limite_diario"}
```

El motor local no interpreta estas reglas, solo traduce cada resultado al mensaje correspondiente (ver tabla de la sección 4): "cooldown" muestra la hora de espera; "limite_diario" se muestra exactamente igual que un rostro no registrado.

Contrato esperado de las 2 APIs externas (D15/D16) — BLOQUEANTE

Esto es lo que este sistema **necesitaría** que cada API externa exponga para que D15 funcione tal cual está planteado — no está confirmado que exista hoy. Mientras no se confirme, la integración real no se construye.

```
// propuesto: POST a la API de clientes de curso, y por separado a la de practicantes
POST /verificar-rostro
  body: {"vector": [0.0123, -0.045, ...], "modelo": "sface-128d"}
  resp (coincide): {"coincide": true, "id": "...", "nombre": "...", "tipo": "cliente_curso"}
  resp (no coincide): {"coincide": false}
```

Como no hay filtro por DNI (RF2), y son 2 sistemas separados (D16), cada reconocimiento necesitaría llamar a **las dos** APIs — salvo que exista o se construya un punto de verificación único que cubra ambas poblaciones. Esto también está pendiente de confirmar.

Autenticación motor local → backends

Cabecera X-Device-Key con una clave estática por dispositivo (mismo patrón ya usado en despliegues anteriores del equipo para autenticar servicios internos), tanto hacia el backend propio como hacia las 2 APIs externas una vez definidas. Credenciales nunca en texto plano — ver sección 7 (D14 introduce un riesgo real al ser una PC compartida).

7. SEGURIDAD, PRIVACIDAD Y RESILIENCIA

Privacidad: por la red solo viajan vectores — nunca fotografías ni video. Como no hay caché local (D15), este sistema nunca almacena de forma persistente ningún dato de referencia biométrica — no hay nada centralizado que robar. Toda comunicación usa TLS.

Riesgo real: PC compartida (D14). A diferencia de un kiosk dedicado y bloqueado, esta PC también la usa la recepcionista para su propio trabajo — eso es superficie de ataque adicional. Medidas recomendadas: cuenta de sistema operativo separada y restringida para este proceso; credenciales de API nunca en texto plano (variable de entorno o almacén de secretos del SO); políticas que limiten qué se puede instalar/ejecutar en esa PC.

Sin resiliencia offline para el reconocimiento: con D15, sin conexión a las 2 APIs externas este sistema no puede identificar a nadie — no hay caché de respaldo. Lo que sí se mantiene: si la verificación tuvo éxito pero falla el registro del evento en el backend propio, ese evento se encola localmente y se reintenta con backoff exponencial.

Conflicto de estado: como el backend propio —nunca el motor local— decide entrada/salida (sección 1), un evento encolado se resuelve correctamente al reenviarse, incluso si mientras tanto llegaron otros eventos de la misma persona por otro medio.

Revocación: si se compromete la clave de un dispositivo, se revoca sin afectar a los demás (ver sección 6).

8. HARDWARE Y REQUISITOS NO FUNCIONALES

Con D14, esto ya no es "selección de un equipo dedicado nuevo" — es verificar que la PC de recepción existente cumpla estos mínimos, y sumarle el segundo monitor.

Componente	Especificación recomendada
CPU	x86-64, 4 núcleos (tipo Intel i3/i5 o equivalente) — a confirmar en la PC real de recepción
RAM	8 GB mínimo, considerando que la PC también corre el trabajo normal de la recepcionista
Almacenamiento	Espacio para modelos + cola local de eventos pendientes (D14) — ya no hay caché de embeddings que guardar
Cámara	USB 1080p, 30 FPS, buen desempeño en baja luz
Monitor 1 (recepcionista)	El que ya usa — sin cambios
Monitor 2 (cliente, RF2/RF11)	15" o más, 1080p, ubicado de cara al cliente escaneado, más alejado que el monitor 1
Red	Conexión estable obligatoria — con D15, sin red no hay reconocimiento posible
Sistema operativo	El que ya tenga la PC de recepción (Windows o Linux) — el motor en Python corre en ambos

Objetivo de latencia: menos de 1 segundo entre "enfoco de 5s completado" y "resultado en pantalla", **una vez resuelto el bloqueante de la sección 1** — el camino crítico ahora incluye 1-2 llamadas de red a las APIs externas, ya no es 100% local.

9. ALCANCE: QUÉ NO DETERMINA ESTE SISTEMA

Dos dudas planteadas por el equipo durante el diseño, resueltas con el mismo criterio de separación de responsabilidades (D11):

¿Asistió a la clase X? (D12) — este sistema solo sabe que la persona estuvo en la instalación entre tal hora y tal hora. No sabe nada de cursos, horarios de clase ni matrícula. Determinarlo exige cruzar sus eventos de entrada/salida con datos que este sistema no tiene — es trabajo de otro proceso, construido sobre los eventos que este sistema expone (sección 6), no del motor de reconocimiento.

¿Llegó temprano o tarde? (D13) — este sistema solo distingue "registrado" / "no registrado" (y, desde RF9/RF10, "cooldown" / "límite diario"). Nunca compara la `marca_tiempo` contra un horario esperado, porque ese horario no está modelado aquí: para practicantes vive en el sistema externo (D9); para clientes del curso, en ningún lugar de este proyecto todavía. Es la misma clase de problema que D12, generalizada: cualquier lectura del evento contra un calendario externo es un concern aparte.

10. PENDIENTES TÉCNICOS EXPLÍCITOS

Este documento propone un diseño técnico detallado a partir de las decisiones ya confirmadas (D1-D18 en docs/PROCESO.md), pero los siguientes puntos siguen abiertos y no deben darse por resueltos:

Bloqueante (confirmado por el equipo): no se avanza en la integración real con los sistemas externos hasta aclarar los 3 puntos siguientes.

- Especificaciones concretas de las 2 APIs externas (autenticación real, endpoints exactos, frecuencia).
- Si exponen o no un endpoint de verificación por vector — de eso depende si D15 (verificación remota, sin caché) es viable tal cual está planteada, o si hay que volver a un modelo de caché local.

- Si hace falta consultar a las 2 APIs en cada intento, o existe/se construye un punto de verificación único para ambas poblaciones (D16).

No bloqueantes, pueden avanzar en paralelo:

- Recalibración del umbral de similitud (0.363 es un punto de partida, no un valor final) con fotografías reales del entorno de BeatC.
- Fuente y formato de los videos publicitarios para RF11 (¿local, streaming, actualizable por BeatC?).
- Pruebas de carga y de latencia real en la PC de recepción (D14), con el camino completo incluyendo las APIs externas.
- Migraciones concretas (Alembic) para el esquema de eventos_asistencia (D18) — la forma ya está decidida, falta implementarla.